

Hybrid Second-Order Optimization of Quantum Circuits: Iterative Preconditioning and Limited-Memory BFGS

by Takudzwa Chitsa

Introduction

Quantum circuits can be designed and composed to construct algorithms that yield specific computational outcomes. However, the implementation of quantum subroutines, such as the Quantum Fourier Transform (QFT) and State Initialization, often necessitates deep quantum circuits, which are challenging to execute on noisy intermediate-scale quantum (NISQ) hardware. These circuits comprise both single-qubit and multi-qubit gate operations, with the latter being particularly prone to noise and decoherence.

To enhance the feasibility of executing these circuits on quantum hardware, it is essential to identify optimal gate sequences that approximate complex unitary transformations with high fidelity while minimizing circuit depth. Additionally, reducing the number of multi-qubit gates is a key objective, as these operations introduce significant errors and contribute to increased noise accumulation. Efficient circuit optimization techniques aim to strike a balance between expressibility and hardware constraints, ultimately improving the practical implementation of quantum algorithms.

Quantum circuit optimization plays a critical role in enhancing the efficiency and accuracy of variational quantum algorithms (VQAs), which are fundamental to near-term quantum computing applications. Traditional first-order methods, such as gradient descent, often suffer from slow convergence and susceptibility to local minima, especially in highly non-convex quantum landscapes. To address these challenges, second-order optimization techniques—leveraging Hessian information—offer a more refined approach to navigating the loss landscape.

In this work, we explore Hybrid Second-Order Optimization strategies for quantum circuits by integrating Iterative Preconditioned Gradient Descent (IPG) and Limited-Memory BFGS (L-BFGS). IPG dynamically updates a preconditioner matrix to approximate second-order curvature information, enabling

more adaptive and efficient updates. L-BFGS, on the other hand, provides an alternative quasi-Newton approach that approximates the Hessian in a memory-efficient manner, making it well-suited for high-dimensional quantum parameter spaces.

Our optimization framework is applied to quantum circuits designed for state preparation and variational quantum eigensolver (VQE) tasks. We analyze the convergence behavior, cost landscapes, and computational performance of both methods, assessing their effectiveness in mitigating barren plateaus and improving fidelity-based cost functions.

Hybrid Optimization

The methodology of the project involves using a classical computer to approximate the rotation angles of the qubit gates. These angles are then fed into a noiseless quantum computing simulator, where the outputs are measured. The measured outputs are compared to the desired states and fed back into the classical computer to make adjustments. This process is repeated multiple times until the desired output state is approximated with high fidelity.

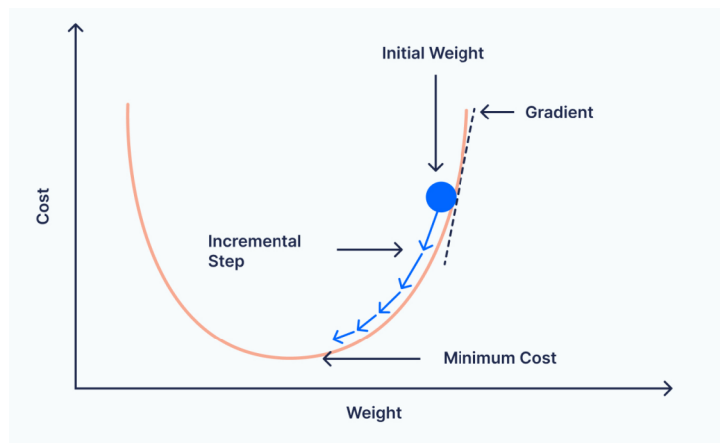
Background

Gradient Based Optimization

First Order Gradient Based Optimization

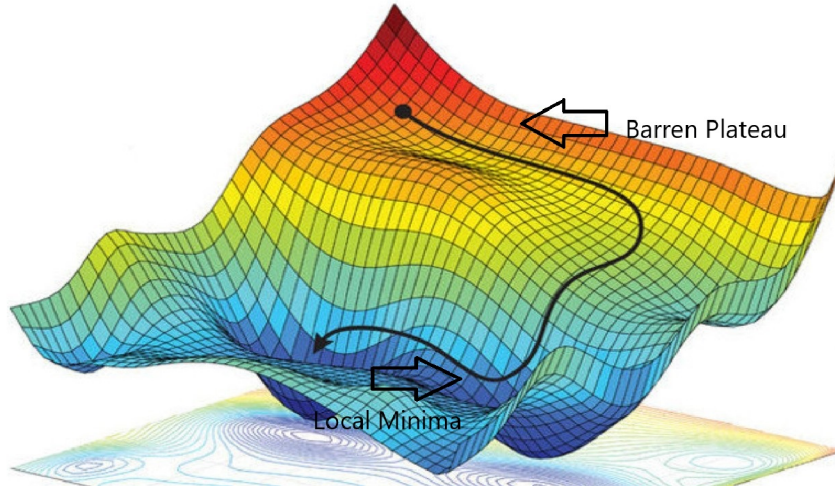
Gradient Descent & Adaptive Moment Estimation (Adam)

- Initial we start with our parameters (initial weight) that we pass into our cost function. Essentially we try to find the parameters that give the minimum values of this cost function by moving in the direction that minimizes the gradient.
- Parameters are updated in step: $\theta_{t+1} = \theta_t - \alpha \frac{\partial}{\partial \theta_t} f(\theta_t)$



Challenge

- Most cost functions contain local minima and barren plateaus. Therefore, if the initial parameters are not close to the global minimum, the true minimum value of the cost function cannot be attained, as they may descend into a local minimum.



High Order Techniques

- Involve computation of the gradient of the cost function
- However, now we also compute the Hessian Matrix

$$H = \nabla^2 f(\theta_{i,j}) = \frac{\partial^2 f}{\partial_i \partial_j}$$

$$\text{Step: } \theta_{t+1} = \theta_t - H^{-1}(\theta_t) \frac{\partial}{\partial \theta_t} f(\theta_t)$$

Challenge

- However, computing the inverse hessian is a very computationally expensive process.

$$O(N^3) + O(N^2)$$

... for hessian computation and inverse calculation

- If there is a large ratio between max and min eigenvalues of the hessian the matrix will be ill-conditioned and produce unpredictable steps.

Solution

Iterative Preconditioned Gradient Descent (IPG)

- This is built upon basic gradient descent
- A notable difference is that the gradients are multiplied by a pre-conditioner matrix. The pre-conditioned gradient updates the current estimate.

Update Step:

- 1) Updating the Parameters

$$x_{t+1} = x_t - \delta_t K_t g_t$$

2) Updating the Preconditioner Matrix

$$K_{t+1} = K_t - \alpha_t((H_t + \beta_t I) K_t - I)$$

where $\alpha_t, \beta_t, \delta_t, t \geq 0$

Conditioning :

$$\delta_t \leq 1, \quad \beta_t > -\lambda_{\min}(H_t), \quad \alpha_t < \frac{1}{\lambda_{\max}(H_t) + \beta_t}$$

where: $\lambda_{\min}(H_t)$ is minimum eigenvalue of H_t & $\lambda_{\max}(H_t)$ is maximum eigenvalue of H_t

Limited-memory Broyden–Fletcher–Goldfarb–Shanno (L-BFGS)

- Quasi-Newton optimization algorithm designed to approximate the inverse Hessian matrix using limited memory.
- In each iteration, the update uses the gradient and parameter differences.

$$s_t = x_t - x_{t-1}$$

$$y_t = \nabla f(x_t) - \nabla f(x_{t-1})$$

Update Step:

$$x_{t+1} = x_t - \alpha_t H^{-1} \nabla f(x_t)$$

- The search direction can be computed via a recursive formula. The two-loop recursion helps compute the direction efficiently:

Step 1:

$$\alpha_i = p_i s_i^T q \quad (\text{for } i = t-1, t-2, \dots, 0)$$

$$q = \nabla f(x_t)$$

$$\text{where } p_i = \frac{1}{y_i^T s_i}$$

Search Direction is then : $r = q$

Step 2:

$$r = r - \sum_{i=0}^{t-1} \alpha_i s_i$$

This gives the search direction (which is the updated gradient direction) :

$$d_t = -r$$

Update Step is thus shown as

$$x_{t+1} = x_t - \alpha_t d_t$$

Optimization

Defining an Ansatz

```
In[*]:= Needs["Wolfram`QuantumFramework`"]

In[*]:=  $\psi[\theta\_]$  := QuantumCircuitOperator[{
  "RZ"[\theta[[1]]] → {1}, "RZ"[\theta[[2]]] → {2}, "RZ"[\theta[[3]]] → {3},
  "RY"[\theta[[4]]] → {1}, "RY"[\theta[[5]]] → {2}, "RY"[\theta[[6]]] → {3},
  "RZ"[\theta[[7]]] → {1}, "RZ"[\theta[[8]]] → {2}, "RZ"[\theta[[9]]] → {3},

  "CNOT" → {1, 2}, "CNOT" → {2, 3}, "CNOT" → {3, 1},

  "RZ"[\theta[[10]]] → {1}, "RZ"[\theta[[11]]] → {2}, "RZ"[\theta[[12]]] → {3},
  "RY"[\theta[[13]]] → {1}, "RY"[\theta[[14]]] → {2}, "RY"[\theta[[15]]] → {3},
  "RZ"[\theta[[16]]] → {1}, "RZ"[\theta[[17]]] → {2}, "RZ"[\theta[[18]]] → {3},

  "CNOT" → {1, 3}, "CNOT" → {2, 1}, "CNOT" → {3, 2}, "RZ"[\theta[[19]]] → {1},

  "RY"[\theta[[20]]] → {1}, "RZ"[\theta[[21]]] → {2}, "RZ"[\theta[[22]]] → {3},
  "RZ"[\theta[[23]]] → {1}, "RY"[\theta[[24]]] → {2}, "RY"[\theta[[25]]] → {3},
  "RZ"[\theta[[26]]] → {2}, "RZ"[\theta[[27]]] → {3},

  "CNOT" → {1, 2}, "CNOT" → {2, 3}, "CNOT" → {3, 1}
}]
```

Visualizing our Ansatz

```
QuantumCircuitOperator[{
  "RZ"[θ] → {1}, "RZ"[θ] → {2}, "RZ"[θ] → {3},
  "RY"[θ] → {1}, "RY"[θ] → {2}, "RY"[θ] → {3},
  "RZ"[θ] → {1}, "RZ"[θ] → {2}, "RZ"[θ] → {3},

  "CNOT" → {1, 2}, "CNOT" → {2, 3}, "CNOT" → {3, 1},

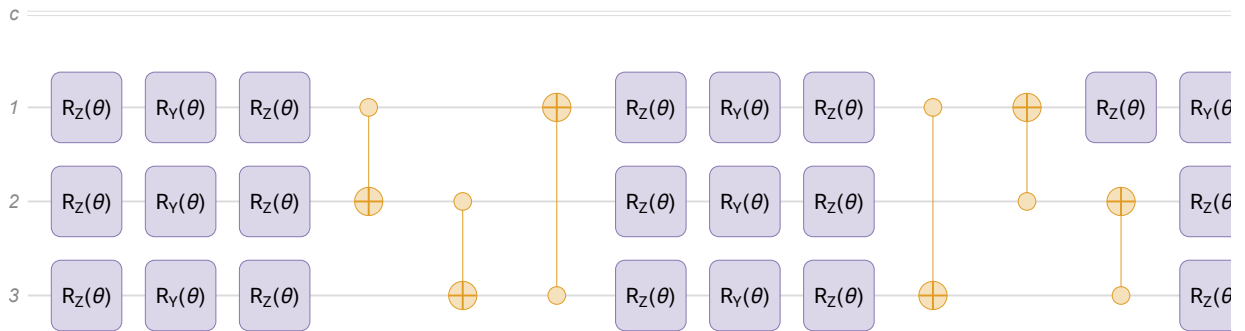
  "RZ"[θ] → {1}, "RZ"[θ] → {2}, "RZ"[θ] → {3},
  "RY"[θ] → {1}, "RY"[θ] → {2}, "RY"[θ] → {3},
  "RZ"[θ] → {1}, "RZ"[θ] → {2}, "RZ"[θ] → {3},

  "CNOT" → {1, 3}, "CNOT" → {2, 1}, "CNOT" → {3, 2}, "RZ"[θ] → {1},

  "RY"[θ] → {1}, "RZ"[θ] → {2}, "RZ"[θ] → {3},
  "RZ"[θ] → {1}, "RY"[θ] → {2}, "RY"[θ] → {3},
  "RZ"[θ] → {2}, "RZ"[θ] → {3},

  "CNOT" → {1, 2}, "CNOT" → {2, 3}, "CNOT" → {3, 1}, {1, 2, 3}
}][ "Diagram"]
```

Out[⌘]=



Initializing Random Parameters

```
θinit = SetPrecision[RandomReal[{-Pi, Pi}, 27], 6]
```

Out[⌘]=

```
{2.76472, -0.154220, 3.08864, 1.46357, 0.293058, 2.48458, -1.16382, -2.88458, 2.56886,
-0.804984, 1.70339, 0.256972, -0.282940, -2.04830, 2.40728, -0.303472, 0.660593, -0.882484,
0.295202, 2.58194, 0.100049, -2.19212, -0.0529144, 2.03330, 2.91949, -2.49875, 3.13611}
```

Defining our Cost Function

$$\text{Cost} = 1 - \text{Re} \langle \Psi_{\text{GHZ} VW} | \Psi_{\text{Output}} \rangle$$

```

In[*]:= Cost[θ_] := Block[{ψout, ψtarget, cost, innerProduct},
  (*Define the target state, initially set to the 3-qubit GHZ state*)
  ψtarget = ψghz;
  (*Compute the output state from the quantum circuit given the parameters θ*)
  ψout = ψ[θ][ψinit]["StateVector"] // Normal;
  (*Compute the inner product between the target state and the output state*)
  innerProduct = ConjugateTranspose[ψtarget].ψout;
  (*Define the cost function as 1 - |Re(innerProduct)| to measure fidelity*)
  cost = 1 - Abs[Re[innerProduct]];

  (*Return the computed cost value*)
  cost]

```

Gradient Function (Using Parameter Shifting)

```

In[*]:= gradient[Cost_, θ_List] := Module[{shift, grad, numParams = Length[θ]},
  (*Define the small shift used for finite difference approximation*) shift = π / 2;
  (*Compute the gradient for each parameter using the finite difference method*)
  grad = Table[
    (*Central difference approximation: (Cost(θ+shift) - Cost(θ-shift)) / (2*shift)*)
    (Cost[ReplacePart[θ, i → θ[[i]] + shift]] - Cost[ReplacePart[θ, i → θ[[i]] - shift]]) / 2,
    (*Iterate over all parameters in θ*) {i, numParams}];
  (*Return the computed gradient vector*)
  grad]

```

Hessian Function (Using Parameter Shifting)

```

In[*]:= HessianM[Cost_, θ_List] := Module[{shift, numParams, hessian, e, i, j},
  (*Define a small perturbation for finite difference approximation*) shift = π / 2;
  (*Get the number of parameters in the list θ*)
  numParams = Length[θ];
  (*Initialize an empty Hessian matrix with zeros*)
  hessian = ConstantArray[0, {numParams, numParams}];
  (*Loop over all pairs of parameters (i,j) to compute the Hessian*)
  For[i = 1, i ≤ numParams, i++, For[j = 1, j ≤ numParams, j++,
    (*Create a unit vector e that combines the i-th and j-th unit vectors*)
    e = UnitVector[numParams, i] + UnitVector[numParams, j];
    (*Compute the (i,j)-th element of the Hessian using finite difference method*)
    hessian[[i, j]] = (Cost[θ + shift * e] - Cost[θ + shift * UnitVector[numParams, i] -
      shift * UnitVector[numParams, j]] - Cost[θ - shift * UnitVector[numParams, i] +
      shift * UnitVector[numParams, j]] + Cost[θ - shift * e]) / 4;]];

  (*Return the computed Hessian matrix*)
  hessian
]

```

Optimizers

Gradient Descent (GD)

```

In[*]:= GDUpdate[ $\theta$ _,  $\alpha$ _ : 0.05] := Module[{grad,  $\theta$ New},
  (*Compute the gradient of the cost function with respect to  $\theta$ *)
  grad = gradient[Cost,  $\theta$ ];
  (*Update  $\theta$  by subtracting the product of the learning rate ( $\alpha$ ) and the gradient*)
   $\theta$ New =  $\theta$  -  $\alpha$  * grad;
  (*Ensure  $\theta$ New stays within the range  $[-\pi, \pi]$  by wrapping it around if necessary*)
   $\theta$ New = Mod[ $\theta$ New +  $\pi$ , 2  $\pi$ ] -  $\pi$ ;
  (*Return the updated  $\theta$ *)
   $\theta$ New]

In[*]:= GD[initial $\theta$ _, numIter_,  $\alpha$ _ : 0.05] :=
Module[{ $\theta$  = initial $\theta$ , costHistory, cost, i}, (*Store the initial cost by evaluating
  the cost function at the initial parameters*) costHistory = {Cost[initial $\theta$ ]};
  (*Iterate optimization steps for numIter iterations*)
  For[i = 1, i  $\leq$  numIter, i++,
    (*Perform a gradient descent update on  $\theta$  using the specified learning rate  $\alpha$ *)
     $\theta$  = GDUpdate[ $\theta$ ,  $\alpha$ ];
    (*Compute the cost at the updated  $\theta$ *) cost = Cost[ $\theta$ ];
    (*Append the current cost to the cost history list*)
    AppendTo[costHistory, cost];];
  (*Return the final optimized  $\theta$  and the history of costs at each iteration*)
  { $\theta$ , costHistory}]

```

```

In[*]:= GDAnalyzer[] := Module[{currentLow, index, costTracker, bestTheta, theta, results,
    optimizedTheta, lowestCost, GDCostTracker, GDavgHistory, GDbestHistory},
  (*Initialize the current lowest cost and index variables*)
  currentLow = 1;
  index = -1;
  (*Initialize an empty list to track cost histories*)
  costTracker = {};
  (*Perform gradient descent 4 times with different random initializations*)
  Do[(*Generate a random initial  $\theta$  within the range  $[-\pi, \pi]$ *)
    theta = RandomReal[{-Pi, Pi}, 27];
    (*Perform gradient descent on the random initial  $\theta$ , with 100 iterations*)
    results = GD[theta, 100];
    (*Extract the optimized  $\theta$  and the cost history from the results*)
    optimizedTheta = results[[1]];
    (*Find the minimum cost from the cost history*)
    lowestCost = Min[results[[2]]];
    (*Update currentLow and index if the new lowest cost is better*)
    If[lowestCost < currentLow, currentLow = lowestCost;
      index = i;
      bestTheta = optimizedTheta;];
    (*Append the cost history of the current run to the costTracker*)
    AppendTo[costTracker, results[[2]]], {i, 4}];
  (*Repeat the above steps 4 times*)
  (*Store the cost history for the run with the lowest cost*)
  GDCostTracker = costTracker;
  (*Compute the average cost history over all runs*)
  GDavgHistory = Mean /@ Transpose[costTracker];
  (*Get the cost history of the run that achieved the lowest cost*)
  GDbestHistory = costTracker[[index]];
  (*Return the best cost history, average cost history, optimized  $\theta$ ,
  and current lowest cost*) {GDbestHistory, GDavgHistory, optimizedTheta, currentLow}]

```

Adaptive Moment estimation (Adam)

```

In[*]:= AdamUpdate[ $\theta$ _, mt0_, vt0_, i_,  $\alpha$ _ : 0.1,  $\beta$ 1_ : 0.9,  $\beta$ 2_ : 0.999,  $\epsilon$ _ : 10^-5] :=
Module[{grad, mt1, vt1, m, v,  $\theta$ New},
  (*Compute the gradient of the cost function with respect to  $\theta$ *)
  grad = gradient[Cost,  $\theta$ ];
  (*Update the moment estimates (first moment and second moment)*)
  mt1 =  $\beta$ 1 * mt0 + (1 -  $\beta$ 1) * grad;
  (*Update first moment estimate (mt)*)
  vt1 =  $\beta$ 2 * vt0 + (1 -  $\beta$ 2) * grad^2;
  (*Update second moment estimate (vt)*) (*Compute the bias-corrected moment estimates*)
  m = mt1 / (1 -  $\beta$ 1^i); (*Correct bias for first moment estimate*)
  v = vt1 / (1 -  $\beta$ 2^i);
  (*Correct bias for second moment estimate*)
  (*Update the parameters  $\theta$  using the Adam update rule*)
   $\theta$ New =  $\theta$  - ( $\alpha$  * m) / (Sqrt[v] +  $\epsilon$ );
  (*Return the updated  $\theta$ , along with the new moment estimates*)
  { $\theta$ New, mt1, vt1}]

In[*]:= Adam[initial $\theta$ _, numIter_,  $\alpha$ _ : 0.01,  $\beta$ 1_ : 0.9,  $\beta$ 2_ : 0.999,  $\epsilon$ _ : 10^-5] :=
Module[{m = 0, v = 0,  $\theta$  = initial $\theta$ , costHistory, cost, i},
  (*Store the initial cost by evaluating the cost function at the initial parameters*)
  costHistory = {Cost[initial $\theta$ ]};
  (*Iterate optimization steps for numIter iterations*)
  For[i = 1, i  $\leq$  numIter, i++,
    (*Perform Adam update for the current parameters  $\theta$ , with moment estimates m and v*)
    { $\theta$ , m, v} = AdamUpdate[ $\theta$ , m, v, i,  $\alpha$ ,  $\beta$ 1,  $\beta$ 2,  $\epsilon$ ];
    (*Compute the cost at the updated  $\theta$ *)
    cost = Cost[ $\theta$ ];
    (*Append the current cost to the cost history list*)
    AppendTo[costHistory, cost];
  ]
  (*Return the final optimized  $\theta$  and the history of costs at each iteration*)
  { $\theta$ , costHistory}]

```

```

In[*]:= AdamAnalyzer[] := Module[{currentLow, index, costTracker, theta, results, optimizedTheta,
  lowestCost, bestTheta, AdamCostTracker, AdamAvgHistory, AdamBestHistory},
  (*Initialize variables to track the lowest cost,
  index of the best run, and the cost history*)
  currentLow = 1;
  (*Set the initial lowest cost to a high value*)
  index = -1;
  (*Initialize the index of the best run*)
  costTracker = {};
  (*List to store the cost history of each run*)
  (*Perform 2 iterations of Adam optimization with different random initializations*)
  Do[
    (*Generate a random initial  $\theta$  within the range  $[-\pi, \pi]$ *)
    theta = RandomReal[{-Pi, Pi}, 27];
    (*Perform Adam optimization on the random initialization for 100 iterations*)
    results = Adam[theta, 100];
    (*Extract the optimized  $\theta$  and the cost history of the current run*)
    optimizedTheta = results[[1]];
    (*Find the minimum cost from the cost history of the current run*)
    lowestCost = Min[results[[2]]];
    (*If the current run's lowest cost is better than the previous lowest,
    update currentLow and bestTheta*) If[lowestCost < currentLow, currentLow = lowestCost;
    (*Update the current lowest cost*)
    index = i; (*Update the index of the best run*)
    bestTheta = optimizedTheta; (*Store the optimized  $\theta$  for the best run*)];
    (*Append the cost history of the current run to the costTracker list*)
    AppendTo[costTracker, results[[2]]];, {i, 2}];
  (*Run the loop 2 times with different random initializations*)
  (*Store the cost history for all runs*)
  AdamCostTracker = costTracker;
  (*Compute the average cost history across all runs at each iteration*)
  AdamAvgHistory = Mean /@ Transpose[costTracker];
  (*Retrieve the cost history of the run with the lowest cost*)
  AdamBestHistory = costTracker[[index]];
  (*Return the best cost history, average cost history, best optimized  $\theta$ ,
  and the lowest cost found*) {AdamBestHistory, AdamAvgHistory, bestTheta, currentLow}]

```

Iterative Preconditioned Gradient Descent (IPG)

```

In[*]:= IPGUpdate[θ_, K_, α_, β_, δ_] := Module[{grad, hessian, I, θNew, KNew},
  (*Compute the gradient of the cost function at the current θ*) grad = gradient[Cost, θ];
  (*Compute the Hessian matrix of the cost function at the current θ*)
  hessian = HessianM[Cost, θ];
  (*Regularize the Hessian matrix by adding a small value to the diagonal to
  ensure it is invertible*) hessian = hessian + 10-23 IdentityMatrix[Length[θ]];
  (*Create an identity matrix of the same size as θ*)
  I = IdentityMatrix[Length[θ]];
  (*Compute the updated θ using the preconditioned gradient descent formula*)
  θNew = θ - δ K.grad;
  (*Normalize the updated θ to ensure it stays within the valid range, [-π, π]*)
  θNew = Mod[θNew + π, 2 π] - π;
  (*Update the preconditioner matrix K using the gradient and Hessian information*)
  KNew = K - α ((hessian + β I).K - I);
  (*Return the updated θ and the updated preconditioner matrix K*)
  {θNew, KNew}]

```

```

In[*]:= IPG[initialθ_, numIterations_ : 50, δ_ : 0.999, ξ_ : 0.01] :=
Module[{H, eigenvalues, λmax, λmin, gradE, norm, β, oδ, α, costHistory, θ, K, cost, i},
  (*Compute Hessian matrix at the initial parameters*)
  H = HessianM[initialθ];
  (*Compute the eigenvalues of the Hessian matrix*)
  eigenvalues = Eigenvalues[H];
  (*Get the maximum and minimum eigenvalues of the Hessian*)
  λmax = Max[eigenvalues];
  λmin = Min[eigenvalues];
  (*Compute the gradient at the initial parameters*)
  gradE = gradient[initialθ];
  (*Calculate the norm of the gradient*)
  norm = Norm[gradE];
  (*Compute the hyperparameters β and oδ*)
  β = ξ + (-λmin);
  (*The regularization term based on the minimum eigenvalue*)
  oδ = norm / (λmax + β);
  (*Update step size adjustment based on eigenvalues*) (*Compute the learning rate α*)
  α = (1 / (λmax + β)) - ξ;
  (*The learning rate based on eigenvalue properties*) (*Initialize the
    cost history list with the initial cost*) costHistory = {Cost[initialθ]};
  (*Initialize parameters θ and preconditioner K*) θ = initialθ;
  K = RandomReal[NormalDistribution[], {Length[θ], Length[θ]}];
  (*Perform optimization for a specified number of iterations*)
  For[i = 1, i ≤ numIterations, i++,
    (*Update θ and preconditioner K using the IPG update rule*)
    {θ, K} = IPGUpdate[θ, K, α, β, δ];
    (*Compute the current cost at the updated θ*)
    cost = Cost[θ];
    (*Append the current cost to the cost history*)
    AppendTo[costHistory, cost];];
  (*Return the optimized parameters θ and the history of cost values over iterations*)
  {θ, costHistory}]

```

```

In[*]:= IPGAnalyzer[] := Module[
  {avgTracker, bestTracker, parameters, theta, delta, zeta, currentLow, index, costTracker,
   results, lowestCost, optimizedTheta, bestTheta, avgHistory, bestHistory, avg, best},
  (*Initialize lists to store results for average and best cost tracking*)
  avgTracker = {};
  bestTracker = {};
  (*Define the parameter grid for delta and zeta values to explore*)
  parameters = <|"delta" → {0.999}, "zeta" → {0.001, 0.01}|>;
  (*Loop over different delta and zeta values to analyze the IPG performance*)
  Do[(*Generate a random initial parameter set  $\theta$  within the range  $[-\pi, \pi]$ *)
    theta = RandomReal[{-Pi, Pi}, 27];
    (*Loop over different zeta values*)
    Do[
      (*Initialize tracking variables for each run*)
      currentLow = 1;
      index = -1;
      costTracker = {};
      (*Loop over multiple trials (2 in this case) for each (delta,zeta) combination*)
      Do[
        (*Run IPG with the current parameters and store the result*)
        results = IPG[ $\theta$ , 100, delta, zeta];
        (*Get the lowest cost from the results and check if it is the lowest so far*)
        lowestCost = Min[results[[2]]];
        optimizedTheta = results[[1]];
        (*Update the best theta and cost tracking if a new lower cost is found*)
        If[lowestCost < currentLow, currentLow = lowestCost;
          index = i;
          bestTheta = optimizedTheta;];
        (*Append the cost history for the current trial*)
        AppendTo[costTracker, results[[2]]];, {i, 2}];
      (*Calculate the average cost
       over all trials for the current (delta,zeta) combination*)
      avgHistory = Mean /@ Transpose[costTracker];
      (*Append the average cost history for later analysis*)
      AppendTo[avgTracker, avgHistory];
      (*Append the best cost history for later analysis*)
      AppendTo[bestTracker, costTracker[[index]]];, {zeta, parameters["zeta"]}],
      {delta, parameters["delta"]}],
    (*Compute the overall average and best cost histories across all experiments*)
    avg = Mean /@ Transpose[avgTracker];
    best = Mean /@ Transpose[bestTracker];
    (*Return the best and average cost histories, as well as the best parameter set found*)
    {best, avg, bestTheta}]

```

Limited-memory Broyden–Fletcher–Goldfarb–Shanno (L-BFGS)

```

In[*]:= BacktrackingLineSearch[Cost_, grad_,  $\theta$ _, searchDir_,  $\alpha$ _ : 1.0,  $\beta$ _ : 0.5, c_ : 10^-5] :=
Module[{ $\alpha$ New =  $\alpha$ }, (*Start with initial alpha value*)
While[
  (*Check if the cost function with the updated theta is greater than the current cost
  compared to the sufficient decrease condition using the gradient and alpha*)
  Cost[ $\theta$  +  $\alpha$ New * searchDir] > Cost[ $\theta$ ] + c  $\alpha$ New (grad.searchDir),
  (*If the condition is not satisfied, reduce alpha by multiplying by beta*)
   $\alpha$ New *=  $\beta$ ];
(*Return the new value of alpha that satisfies the condition*)
 $\alpha$ New]

```

```

In[*]:= LBFGS[costFunction_, gradCompute_,  $\theta$ init_, m_:10, maxIter_:40, tol_:10-6] :=
Module[{ $\theta$  =  $\theta$ init, grad, costHistory, sList, yList,  $\rho$ List, k, q,  $\alpha$ List,
  i,  $\alpha$ ,  $\gamma$ , r,  $\beta$ , searchDir,  $\theta$ new, gradNew, s, y,  $\rho$ }, grad = gradient[ $\theta$ ];
costHistory = {costFunction[ $\theta$ ]};

(*Storage for past updates*) sList = {};
(*Parameter differences*) yList = {};
(*Gradient differences*)  $\rho$ List = {};
(*Scaling factors*) (*Optimization Loop*) For[k = 1, k ≤ maxIter, k++,
  (*Convergence check*) If[Norm[grad] < tol, Print["Converged!"];
  Break[]];
  (*Two-loop recursion for search direction*) q = grad;
   $\alpha$ List = {};
  For[i = Length[sList], i ≥ 1, i--,  $\alpha$  =  $\rho$ List[[i]].(sList[[i]].q);
  AppendTo[ $\alpha$ List,  $\alpha$ ];
  q = q -  $\alpha$  * yList[[i]];];
  (*Scale initial Hessian approximation*)
  If[Length[sList] > 0,  $\gamma$  = (sList[[-1]].yList[[-1]]) / (yList[[-1]].yList[[-1]] + 10-8);
  r =  $\gamma$  * q, r = q];
  For[i = 1, i ≤ Length[sList], i++,  $\beta$  =  $\rho$ List[[i]].(yList[[i]].r);
  r = r + sList[[i]] * ( $\alpha$ List[[Length[sList]] - i + 1] -  $\beta$ );];
  searchDir = -r;
  (*Perform backtracking line search*)
   $\alpha$  = BacktrackingLineSearch[costFunction, grad,  $\theta$ , searchDir];
   $\theta$ new =  $\theta$  +  $\alpha$  * searchDir;
  gradNew = gradient[ $\theta$ new];
  (*Update history*) s =  $\theta$ new -  $\theta$ ;
  y = gradNew - grad;
   $\rho$  = 1 / (y.s + 10-8);
  AppendTo[sList, s];
  AppendTo[yList, y];
  AppendTo[ $\rho$ List,  $\rho$ ];
  (*Maintain memory size*) If[Length[sList] > m, sList = Drop[sList, 1];
  yList = Drop[yList, 1];
   $\rho$ List = Drop[ $\rho$ List, 1];];
  (*Update for next iteration*)  $\theta$  =  $\theta$ new;
  grad = gradNew;
  AppendTo[costHistory, costFunction[ $\theta$ ]];];
{ $\theta$ , costHistory}]

```

```

In[*]:= lbfgsAnalyzer[] :=
Module[{currentLow, index, costTracker, bestTheta, theta, results, optimizedTheta,
  lowestCost, lbfgsCostTracker, lbfgsAvgHistory, lbfgsBestHistory}, (*Initialize
  variables to track the lowest cost and the best solution found*)currentLow = 1;
index = -1;
costTracker = {};
(*Perform multiple trials with random initializations for theta*)
Do[(*Generate a random initialization of theta in the range[-Pi,Pi]*)
  theta = RandomReal[{-Pi, Pi}, 27];
  (*Perform LBFGS optimization starting from the current theta*)
  results = LBFGS[Cost, gradient, theta];
  (*Extract the optimized theta and the history of costs from the results*)
  optimizedTheta = results[[1]];
  lowestCost = Min[results[[2]]];
  (*Check if the current trial resulted in a lower cost than the previous best*)
  If[lowestCost < currentLow, currentLow = lowestCost;
    index = i;
    bestTheta = optimizedTheta;];
  (*Append the cost history of this trial to the overall cost tracker*)
  AppendTo[costTracker, results[[2]]];, {i, 2}];
(*Run the optimization twice for different random initializations*)
(*Store the full cost history across all trials*)lbfgsCostTracker = costTracker;
(*Compute the average cost history over all trials*)
lbfgsAvgHistory = Mean /@ Transpose[costTracker];
(*Extract the best cost history (the one with the lowest cost found)*)
lbfgsBestHistory = costTracker[[index]];
(*Return the best cost history,
average cost history,optimized theta,and the best theta found*)
{lbfgsBestHistory, lbfgsAvgHistory, optimizedTheta, bestTheta}]

```

Analysis

Visualizing Optimized Parameters & Comparing Convergence

Performance over 1000 epochs

- The 3 qubit GHZ state is the optimization target. Optimization is carried out using 27 rotation gates.
- 100 iteration steps for q = 3 qubits are also chosen to optimize for the W state as this is a harder optimization problem than the GHZ state preparation.

Initializing our target states:3-qubit-GHZ and 3-qubit-W

```
 $\psi_{\text{ghz}} = \text{QuantumState}["\text{GHZ}"][3][\text{"StateVector"}] // \text{Normal}$ 
```

```
 $\psi_{\text{w}} = \text{QuantumState}["\text{W}"][3][\text{"StateVector"}] // \text{Normal}$ 
```

```
Out[ ]:=
```

$$\left\{ \frac{1}{\sqrt{2}}, 0, 0, 0, 0, 0, 0, \frac{1}{\sqrt{2}} \right\}$$

```
Out[ ]:=
```

$$\left\{ 0, \frac{1}{\sqrt{3}}, \frac{1}{\sqrt{3}}, 0, \frac{1}{\sqrt{3}}, 0, 0, 0 \right\}$$

$$|\Psi_{\text{GHZ}}\rangle = (|0\rangle^{\otimes N} \pm |1\rangle^{\otimes N})/\sqrt{2}$$

$$|\Psi_{\text{W}}\rangle = \frac{|00\dots 001\rangle + |00\dots 010\rangle + |00\dots 100\rangle + \dots + |10\dots 000\rangle}{\sqrt{N}}$$

Running the Optimizers

Adam

```
In[ ]:= {ghzAdamBestHistory, ghzAdamAvgHistory, ghzAdamOptTheta, ghzAdamCost} = AdamAnalyzer[  
  (* Target state 3 GHZ*)];
```

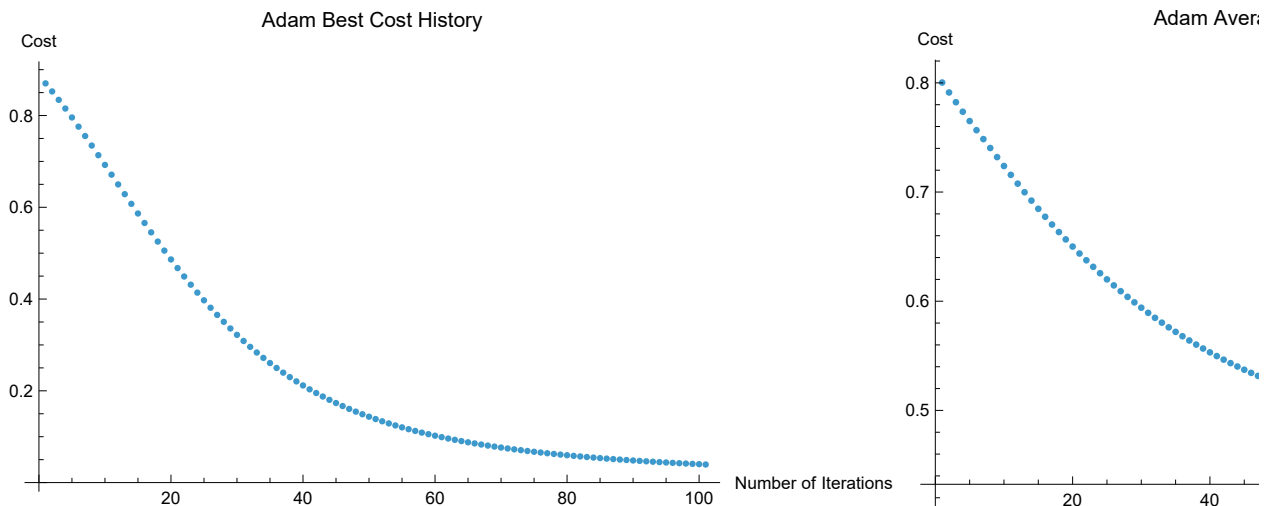
```
In[ ]:= {wAdamBestHistory, wAdamAvgHistory, wAdamOptTheta, wAdamCost} = AdamAnalyzer[  
  (* Target state 3 W (Cost Function Edited)*)];
```

```
In[ ]:= Grid[{{ListPlot[wAdamBestHistory,  
  PlotLabel → "Adam Best Cost History", AxesLabel → {"Number of Iterations", "Cost"}},  
  ListPlot[wAdamAvgHistory, PlotLabel → "Adam Average Cost History",  
  AxesLabel → {"Number of Iterations", "Cost"}}}}]
```

```
Grid[{{ListPlot[ghzAdamBestHistory,  
  PlotLabel → "Adam Best Cost History", AxesLabel → {"Number of Iterations", "Cost"}},  
  ListPlot[ghzAdamAvgHistory, PlotLabel → "Adam Average Cost History",  
  AxesLabel → {"Number of Iterations", "Cost"}}}}]
```

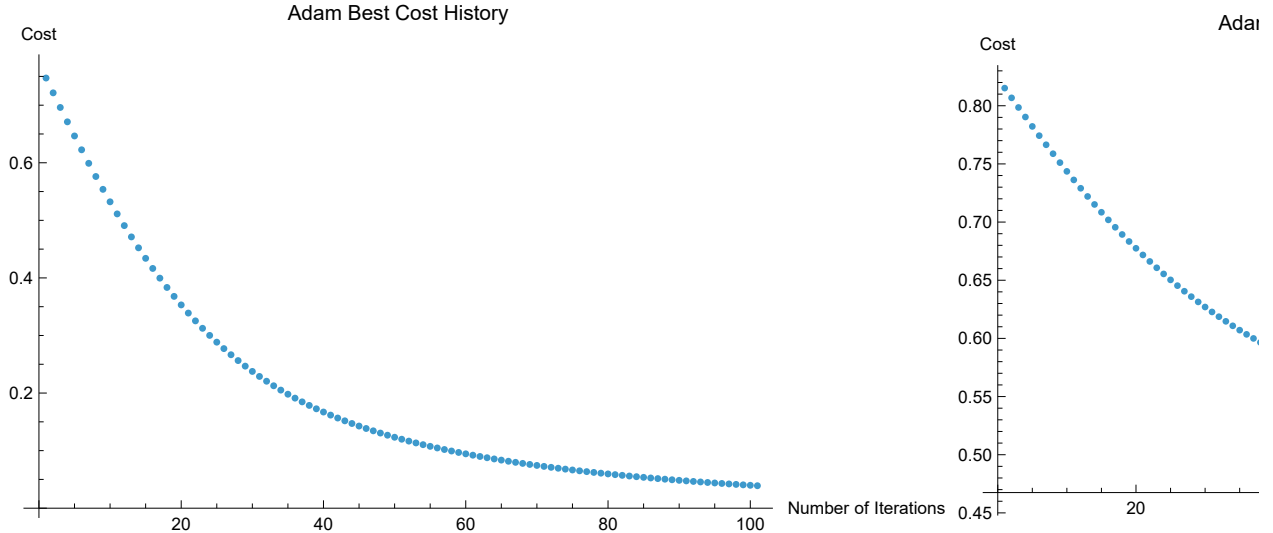
3 Qubit W

```
Out[ ]:=
```



3 Qubit GHZ

Out[]=



Gradient Descent

```

In[ ]:= {ghzGDBestHistory, ghzGDAvgHistory, ghzGDOptTheta, ghzGDCost} = GDAnalyzer[];
(* Target State 3 GHZ*)

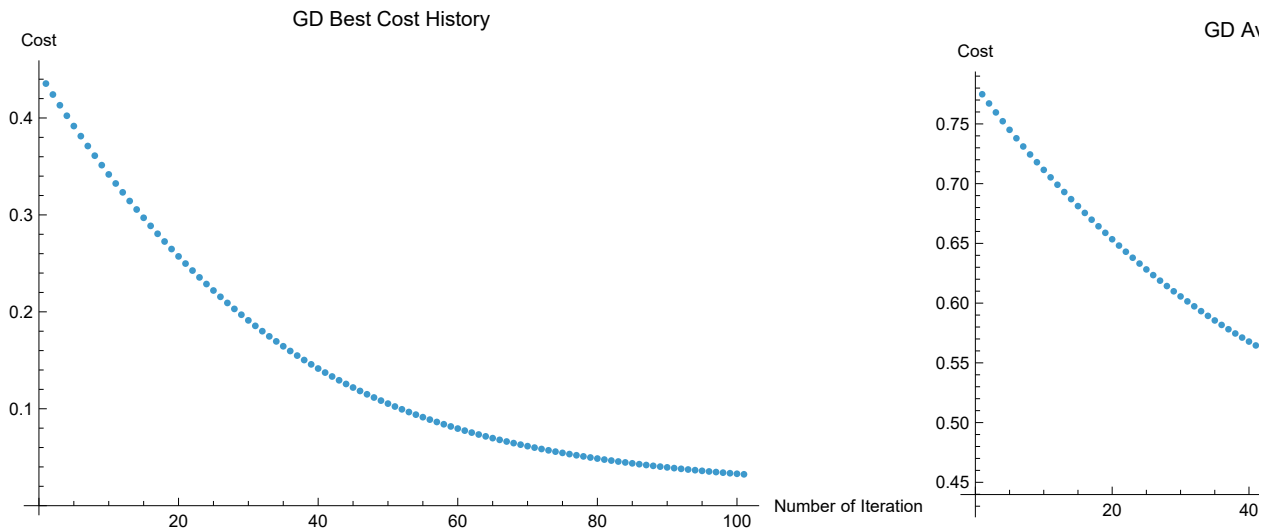
In[ ]:= {wGDBestHistory, wGDAvgHistory, wGDOptTheta, wGDCost} = GDAnalyzer[];
(* Target State 3 W (Cost function edited)*)

In[ ]:= Grid[{{ListPlot[ghzGDBestHistory, PlotLabel → "GD Best Cost History",
  AxesLabel → {"Number of Iteration", "Cost"}], ListPlot[ghzGDAvgHistory,
  PlotLabel → "GD Average Cost History", AxesLabel → {"Number of Iteration", "Cost"}]}]}]
Grid[{{ListPlot[wGDBestHistory, PlotLabel → "GD Best Cost History",
  AxesLabel → {"Number of Iteration", "Cost"}], ListPlot[wGDAvgHistory,
  PlotLabel → "GD Average Cost History", AxesLabel → {"Number of Iteration", "Cost"}]}]}]

```

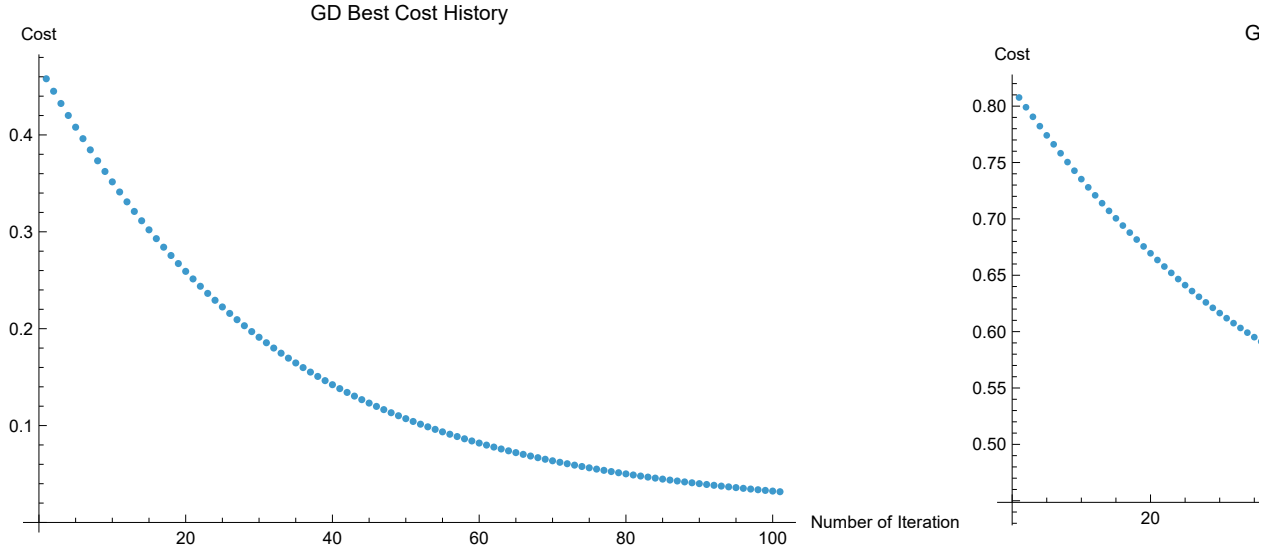
3 Qubit GHZ

Out[]=



3 Qubit W

Out[]:=



IPG

```

In[ ]:= {ghzIPGBestHistory, ghzIPGAvgHistory, ghzIPGOptTheta} = IPGAnalyzer[];
(* Target State 3 GHZ*)

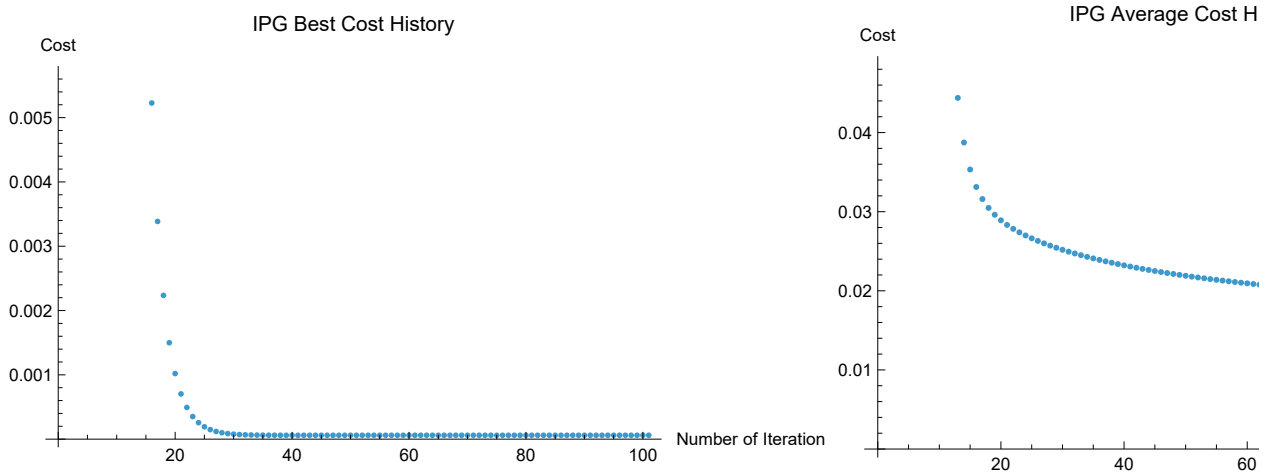
In[ ]:= {wIPGBestHistory, wIPGAvgHistory, wIPGOptTheta} = IPGAnalyzer[];
(* Target State 3 W (Cost function edited)*)

In[ ]:= Grid[{{ListPlot[ghzIPGBestHistory,
  PlotLabel → "IPG Best Cost History", AxesLabel → {"Number of Iteration", "Cost"}],
  ListPlot[ghzIPGAvgHistory, PlotLabel → "IPG Average Cost History",
  AxesLabel → {"Number of Iteration", "Cost"}]}]}]
Grid[{{ListPlot[wIPGBestHistory, PlotLabel → "IPG Best Cost History",
  AxesLabel → {"Number of Iteration", "Cost"}], ListPlot[wIPGAvgHistory,
  PlotLabel → "IPG Average Cost History", AxesLabel → {"Number of Iteration", "Cost"}]}]}]

```

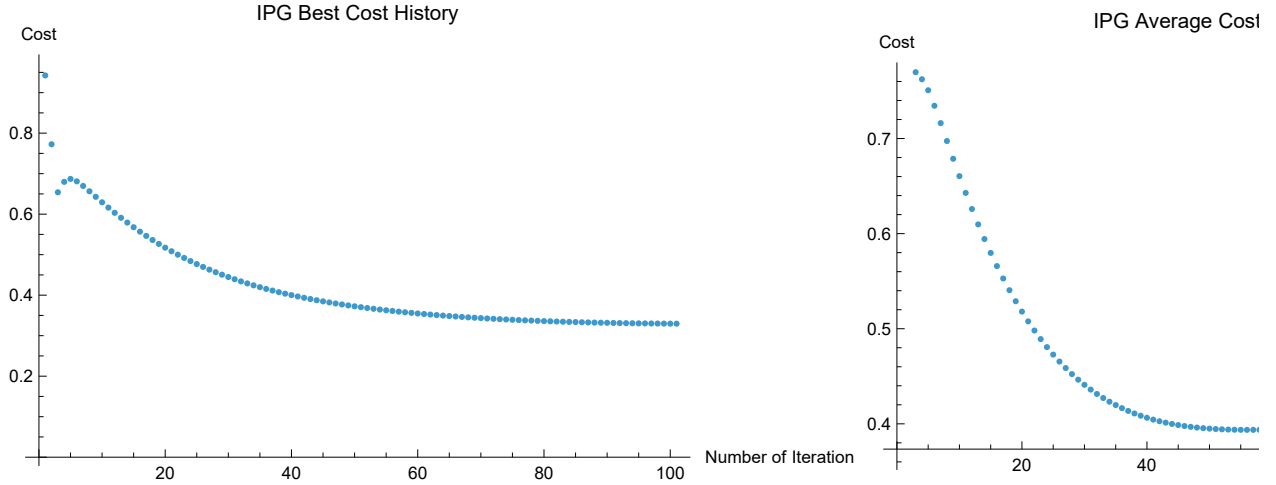
3 Qubit GHZ

Out[]:=



3 Qubit W

Out[]:=



The IPG iteration were run on multiple different non negative scalar parameters, including some that produce poor conditioning.

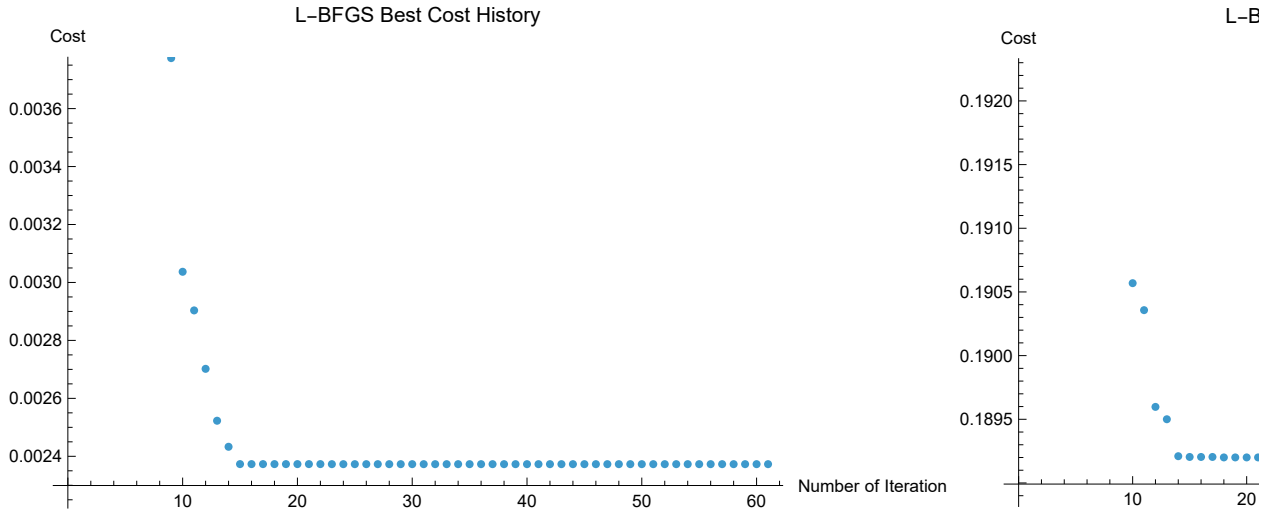
L-BFGS

```
In[ ]:= {wlbfgsBestHistory, wlbfgsAvgHistory, wlbfgsOptTheta, wlbfgsCost} = lbfgsAnalyzer[];
(* Target State 3 W (Cost function edited)*)

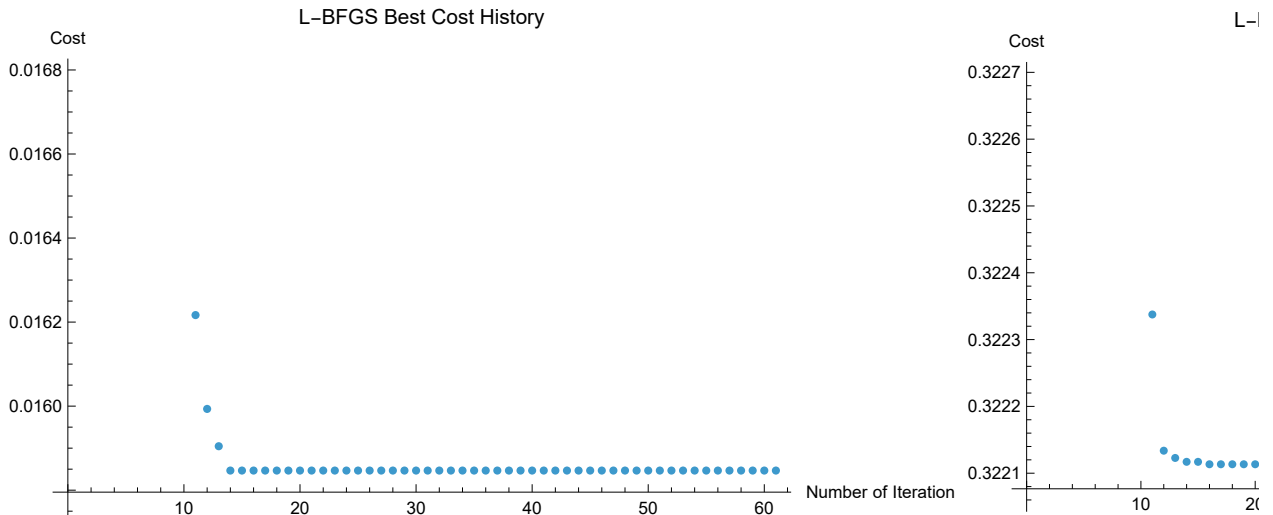
In[ ]:= {ghzlbfgsBestHistory, ghzlbfgsAvgHistory, ghzlbfgsOptTheta, ghzlbfgsCost} =
  lbfgsAnalyzer[]; (* Target State 3 GHZ*)

In[ ]:= Grid[{{ListPlot[ghzlbfgsBestHistory,
  PlotLabel → "L-BFGS Best Cost History", AxesLabel → {"Number of Iteration", "Cost"}],
  ListPlot[ghzlbfgsAvgHistory, PlotLabel → "L-BFGS Average Cost History",
  AxesLabel → {"Number of Iteration", "Cost"}]}]}]
Grid[{{ListPlot[wlbfgsBestHistory,
  PlotLabel → "L-BFGS Best Cost History", AxesLabel → {"Number of Iteration", "Cost"}],
  ListPlot[wlbfgsAvgHistory, PlotLabel → "L-BFGS Average Cost History",
  AxesLabel → {"Number of Iteration", "Cost"}]}]}]
```

3 Qubit GHZ

Out[*n*]=

3 Qubit W

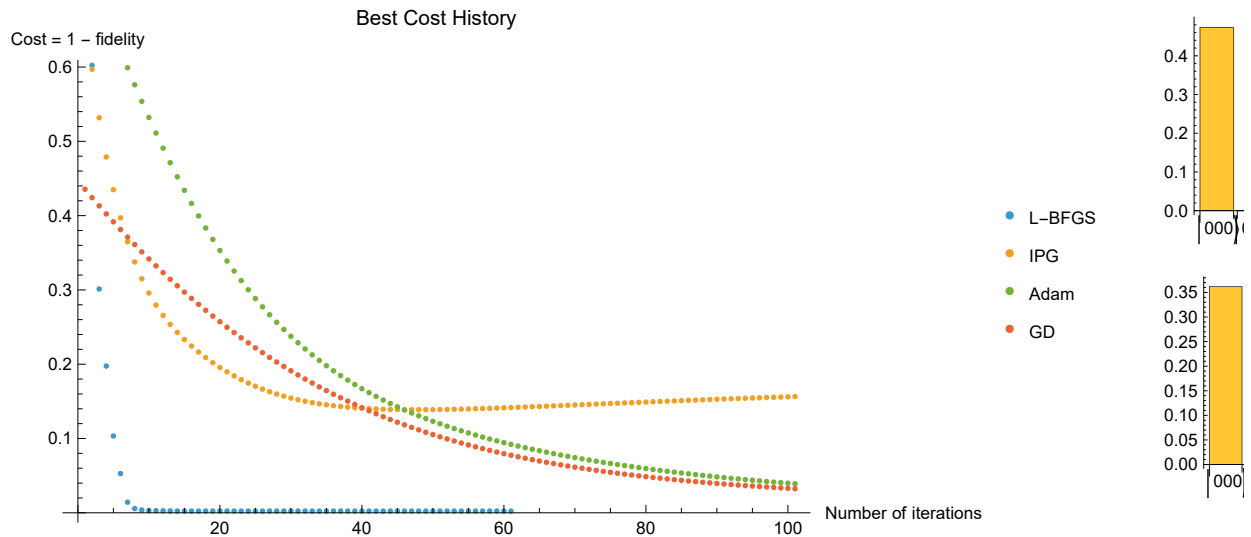
Out[*n*]=

Comparison

3 Qubit GHZ

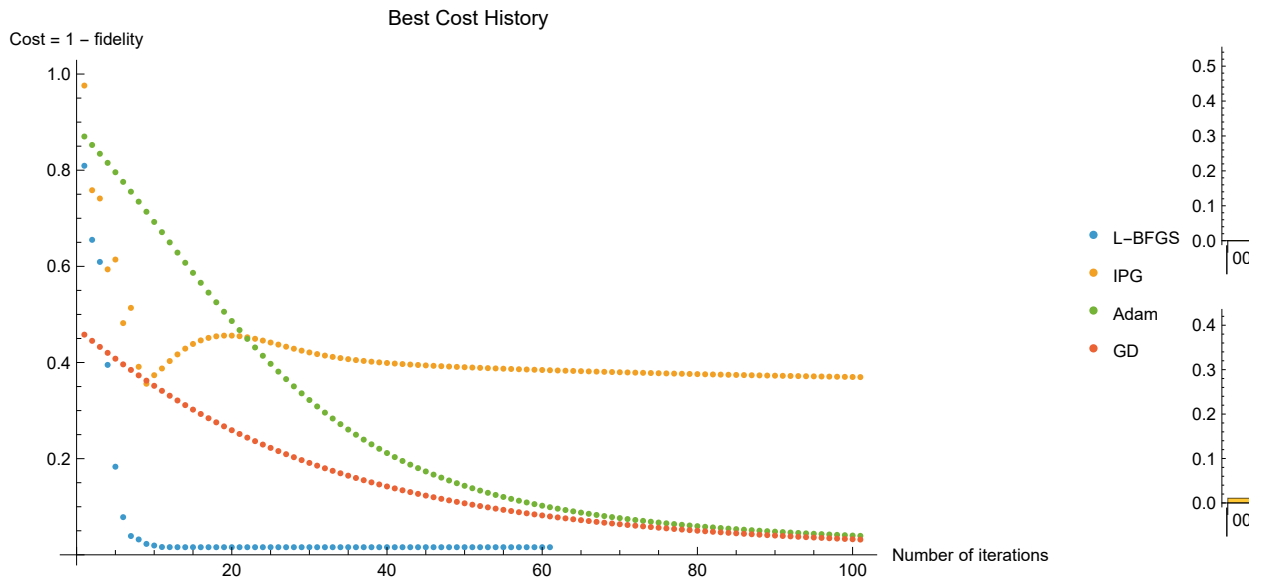
In[*n*]:=

```
Graphics[{Inset[ListPlot[{ghzlbfgsBestHistory, ghzipgbestHistory,
  ghzadamBestHistory, ghzgdbestHistory}, PlotLabel -> "Best Cost History",
  AxesLabel -> {"Number of iterations", "Cost = 1 - fidelity"},
  PlotLegends -> {"L-BFGS", "IPG", "Adam", "GD"}], {0, 0}, {Center, Center}, Scaled[5]],
  Inset[Grid[{{ψ[ghzlbfgsoptTheta][["ProbabilityPlot", PlotLabel -> "L-BFGS"]],
    ψ[ghzipgoptTheta][["ProbabilityPlot", PlotLabel -> "IPG"]],
    {ψ[ghzadamoptTheta][["ProbabilityPlot", PlotLabel -> "Adam"]],
    ψ[ghzgdoptTheta][["ProbabilityPlot", PlotLabel -> "GD"]]}},
    {0, 0}, {Left, Center}, Scaled[0.8]}], AspectRatio -> Full]
```



3 Qubit W State

```
In[ ]:= Graphics[
  {Inset[ListPlot[{wlbfgsBestHistory, wIPGBestHistory, wAdamBestHistory, wGDBestHistory},
    PlotLabel → "Best Cost History",
    AxesLabel → {"Number of iterations", "Cost = 1 - fidelity"},
    PlotLegends → {"L-BFGS", "IPG", "Adam", "GD"}], {0, 0}, {Center, Center}, Scaled[5]],
  Inset[Grid[{ψ[wlbfgsOptTheta][["ProbabilityPlot", PlotLabel → "L-BFGS"],
    ψ[wIPGOptTheta][["ProbabilityPlot", PlotLabel → "IPG"]],
    {ψ[wAdamOptTheta][["ProbabilityPlot", PlotLabel → "Adam"],
    ψ[wGDOptTheta][["ProbabilityPlot", PlotLabel → "GD"]]}],
    {0, 0}, {Left, Center}, Scaled[0.8]], AspectRatio → Full]
```



Testing High Order Optimized Parameters on Quantum Computers

```
ibmq = ServiceConnect["IBMQ", "New"]
```

Out[]:=

```
ServiceObject[ IBM Quantum IBMQ ]
```

3 Qubit GHZ State

```
In[ ]:=  $\theta_{\text{ghz}}$  = ghzIPGOptTheta;
```

```
In[ ]:= ghzQiskit =  $\psi[\theta_{\text{ghz}}][["Qiskit"]][\text{"Decompose"}];$ 
```

```
ghzQPY =
```

```
BaseEncode@ghzQiskit["QPY", "Provider" → "IBMPProvider", "Backend" → "ibm_brisbane"];
```

```
In[ ]:= ghzJob = ibmq["RunCircuit", {"QPY" → ghzQPY, "Backend" → "ibm_brisbane"}];
```

```
ghzJobResult = ibmq["JobResults", {"JobID" → ghzJob["id"]}];
```

```
In[ ]:= ghzCounts = ghzJobResult["results", All, "data", "counts"];
```

```
ghzExpResults = Normalize[N@First[Normal[Values[ghzCounts]]], Total]
```

Out[]:=

```
 $\left\{ \frac{22}{47}, 0, 0, \frac{1}{47}, \frac{3}{94}, \frac{1}{94}, 0, \frac{22}{47} \right\}$ 
```

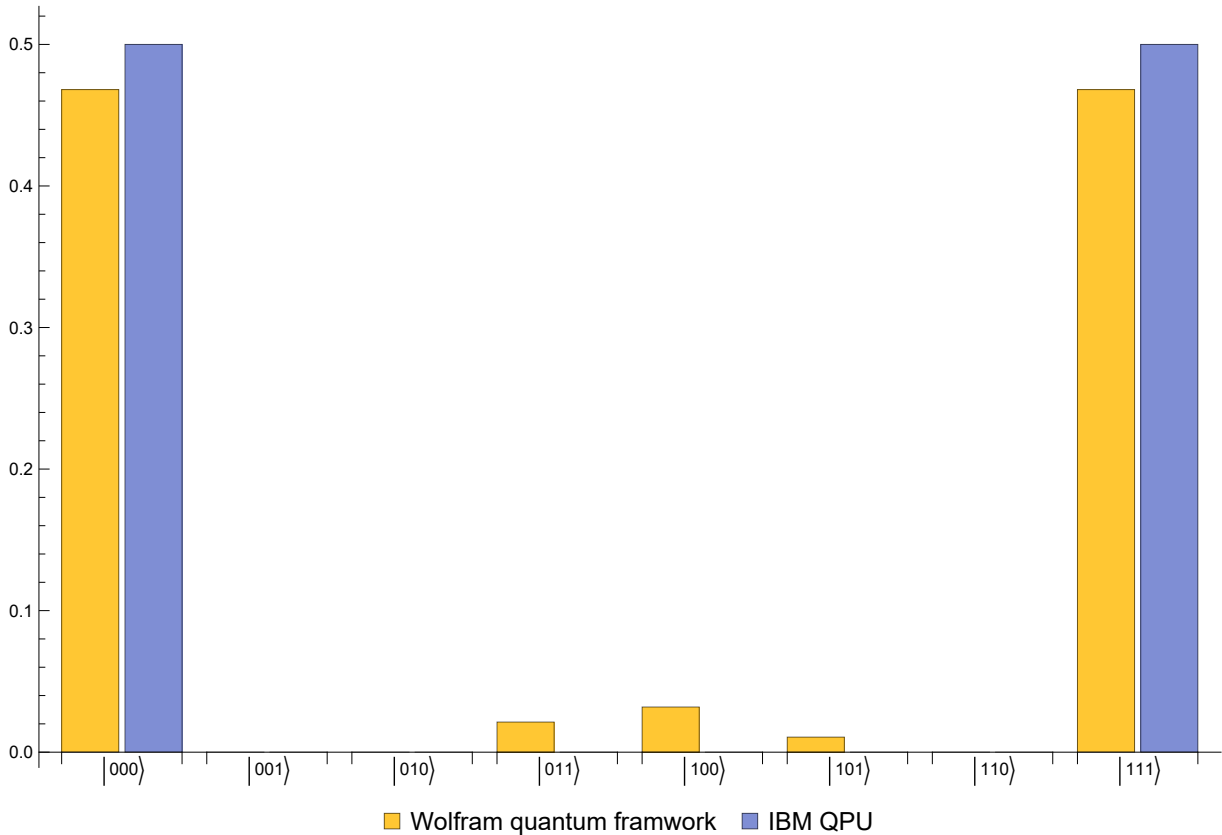
```
ghztheoryResults = QuantumState["GHZ"][3][["ProbabilityList"]] // N
```

```
Out[⌘]=
```

```
{0.5, 0., 0., 0., 0., 0., 0., 0.5}
```

```
BarChart[Transpose[{ghzExpResults, ghztheoryResults}],
  ChartLegends → Placed[{"Wolfram quantum framwork", "IBM QPU"}, Below],
  "ChartLabels" → {Wolfram`QuantumFramework`QuantumState[ghzExpResults][["ElementNames"]],
    None}, ImageSize → Full]
```

```
Out[⌘]=
```



3 Qubit W state

```
In[⌘]:= ⍬w = wIPGOptTheta; (*200 iterations*)
```

```
In[⌘]:= wQiskit = ⍬[⍬w][["Qiskit"]][["Decompose"]];
wQPY = BaseEncode@wQiskit["QPY", "Provider" → "IBMProvider", "Backend" → "ibm_brisbane"];
```

```
In[⌘]:= wJob = ibmq["RunCircuit", {"QPY" → wQPY, "Backend" → "ibm_brisbane"}];
wJobResult = ibmq["JobResults", {"JobID" → wJob["id"]}];
```

```
In[⌘]:= wCounts = wJobResult["results", All, "data", "counts"];
```

```
wExpResults = Normalize[N@First[Normal[Values[wCounts]]], Total]
```

```
Out[⌘]=
```

```
{1/95, 44/95, 22/95, 2/95, 4/19, 1/95, 0, 1/19}
```

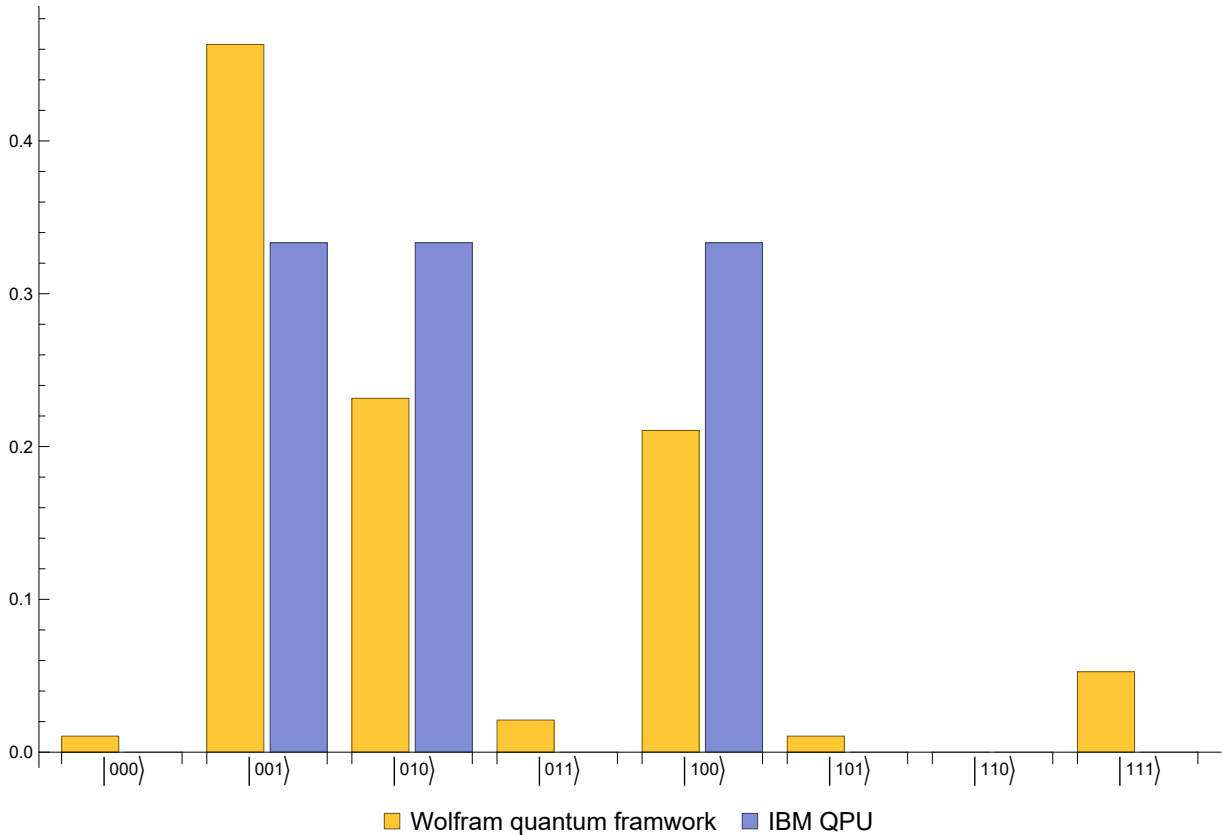
```
wtheoryResults = QuantumState["W"][3][["ProbabilityList"]] // N
```

```
Out[⌘]=
```

```
{0., 0.333333, 0.333333, 0., 0.333333, 0., 0., 0.}
```

```
BarChart[Transpose[{wExpResults, wtheoryResults}],
  ChartLegends → Placed[{"Wolfram quantum framework", "IBM QPU"}, Below],
  "ChartLabels" → {Wolfram`QuantumFramework`QuantumState[wExpResults][["ElementNames"]],
    None}, ImageSize → Full]
```

```
Out[⌘]=
```



Both IPG and L-BFGS achieve high fidelity with the target state at their best iteration while maintaining a sufficiently high average fidelity in the GHZ state over the optimization process. Optimization of the W State did not achieve similar iteration periods but did achieve adequate fidelity over longer iteration periods for IPG and L-BFGS. This suggests that both methods are capable of effectively optimizing quantum circuits, though with different trade-offs in terms of convergence speed and noise suppression.

Conclusion & Discussion

Our results demonstrate that quasi-Newton methods significantly outperform first-order optimization techniques in optimizing quantum circuits, yielding quantum states with substantially reduced noise—by orders of magnitude. Notably, L-BFGS exhibited superior convergence rates compared to Iterative Preconditioned Gradient Descent (IPG). However, despite its slower convergence, IPG produced quan-

tum states with lower noise levels in both the 3-qubit GHZ and W-state preparations.

These findings underscore the efficacy of higher-order derivative methods in quantum circuit optimization, reinforcing their role in improving quantum subroutines. Future analysis will focus on extending these methods to the optimization of full unitary transformations, particularly in scenarios with unknown initial conditions, such as the Quantum Fourier Transform (QFT). Additionally, further investigation into matrix conditioning techniques is warranted to enhance numerical stability and ensure robust convergence across various circuit architectures.

References

1. Quantum Circuit Optimization through Iteratively Pre-Conditioned Gradient Descent: *Dhruv Srivastava, Kushal Chakrabarti, Nikhil Chopra, Avik Dutt* <https://arxiv.org/pdf/2309.09957v1>